Reduction of Qubits in Quantum Algorithm for Monte Carlo Simulation by Pseudo-random Number Generator

Koichi Miyamoto*
Mizuho-DL Financial Technology Co., Ltd.
2-4-1 Kojimachi, Chiyoda-ku, Tokyo, 102-0083, Japan

Kenji Shiohara†
Division of Physics, Graduate School of Science, Hokkaido University
Sapporo, Hokkaido, 060-0810, Japan
(Dated: August 31, 2020)

It is known that quantum computers can speed up Monte Carlo simulation compared to classical counterparts. There are already some proposals of application of the quantum algorithm to practical problems, including quantitative finance. In many problems in finance to which Monte Carlo simulation is applied, many random numbers are required to obtain one sample value of the integrand, since those problems are extremely high-dimensional integrations, for example, risk measurement of credit portfolio. This leads to the situation that the required qubit number is too large in the naive implementation where a quantum register is allocated per random number. In this paper, we point out that we can reduce qubits keeping quantum speed up if we perform calculation similar to the classical one, that is, estimate the average of integrand values sampled by a pseudo-random number generator (PRNG) implemented on a quantum circuit. We present not only the overview of the idea but also concrete implementation of PRNG and application to credit risk measurement. Actually, reduction of qubits is a trade-off against increase of circuit depth. Therefore full reduction might be impractical, but such a trade-off between speed and memory space will be important in adjustment of calculation setting considering machine specs, if large-scale Monte Carlo simulation by quantum computer is in operation in the future.

arXiv:1911.12469v2 [quant-ph] 28 Aug 2020

I. INTRODUCTION

Among applications of quantum computers to numerical problems providing higher speed than classical computation is Monte Carlo simulation [1]. It has been shown that estimation error in the quantum-based Monte Carlo is proportional to $O(N^{-1})$, where N is the number of computational steps, compared with $O(N^{-1/2})$ in the classical one. Quantitative finance is one of the fields where Monte Carlo simulation is heavily used and there are some proposals to apply the quantum algorithm to financial problems, for example, risk measurement of portfolio [2, 3] and derivative pricing [4, 5].

In the application of the quantum algorithm for Monte Carlo to financial problems, required qubit number might be problematic due to the following two points. First, in the methods proposed in the previous works [2–5], a quantum register is allocated to represent a random number, so the required qubit number is proportional to the number of the random numbers required to obtain one sample value of the integrand (in other words, the dimension of the integral). Second, many of the problems in finance are extremely high-dimensional integrations and require many random numbers. One of the most prominent examples is risk measurement of credit portfolio [3]. Credit portfolio consists of many loans or debts and banks suffer losses when obligors default. Banks monitor such credit risks estimating some *risk measures*, for example, expected loss (EL), value-at-risk (VaR), which represents percentile point (say, 99%) of loss distribution, conditional VaR (CVaR), expectation value of loss conditioned it exceeds the VaR, and so on. One of popular mathematical models describing probability distribution of loss is the Merton model [6] and risk measures under the model are usually estimated by Monte Carlo. We describe the model in the later section, but an important point is that the required number of random numbers to determine a default pattern of obligors is nearly equal to the number of obligors. In other words, it is necessary to generate as many random numbers as obligors to obtain a sample value of the integrand, that is, loss. The number of obligors can be $O(10^6)$ for large portfolios, and so is the required random number. The qubit number of today's largest quantum computer is $O(10)$, so it will be the far future when machines with so many qubits are realized. Therefore, it is meaningful to consider the possibility to reduce qubits.

In this paper, we propose a way to reduce qubits. Although we will explain the detail in the next section, we here describe the outline. In short, it is *classical Monte Carlo on a quantum computer*. In classical Monte Carlo, we usually generate *some sampled patterns* of values of random numbers, but *not all patterns*. More concretely, we generate sequences of pseudo-random number (PRN) using some pseudo-random number generator (PRNG) and use each sequence to obtain one sample value of the integrand. Finally, we calculate the average of the sample values and consider it as an approximation of the integral. An important point is that in this way we sequentially generate PRNs and do not require the memory space proportional to the number of the required random numbers. We can do the same thing on a quantum circuit. That is, we can sequentially generate pseudo-random bit strings on a quantum register and calculate the integrand into another register. On a quantum computer, we can parallelly perform such computation and finally obtain the superposition of states in which each of the sampled integrand values is realized on a regis-

* koichi-miyamoto@fintec.co.jp
† k-shiohara@particle.sci.hokudai.ac.jp

ter. Then, we can estimate the average of sample values by the quantum amplitude estimation methods [7, 8], which are commonly used in the quantum algorithm for Monte Carlo. This procedure leads to the same estimation result as classical Monte Carlo, but with quadratic speedup.

We present not only the idea but also concrete implementation. We propose an example of PRNG which can be easily implemented on a quantum circuit. It is *permuted congruential generator (PCG)* [9] and explained in detail in a later section. This is the combination of the linear congruential method and permutation of bit string and has advantages in the aspect of memory (that is, required qubit number) and computational load (that is, circuit depth) compared to other types of PRNG, for example Mersenne Twister [10]. It is possible to construct the quantum gate which progresses the PCG sequence as we present later.

We also consider application to concrete problems. The first one is credit risk measurement, which is mentioned above. We later present the quantum circuit which calculates sampled values of loss of a credit portfolio using PRNG. The second one is the integration of a simple multi-variable function, that is, a trigonometric function whose phase depends on two variables. We consider this for demonstrative purpose and present not only the circuit but also the numerical result calculated by a simulator.

The rest of this paper is organized as follows. Section II explains the overview of our idea. Section III presents concrete implementation of the gate which realizes PCG. In section IV and V we consider application to credit risk measurement and a simple integration, respectively. Section VI contains conclusion and discussion on some issues. Especially, we discuss the trade-off between qubit number and circuit depth and importance of such a memory-speed trade-off on adjustment of calculation configuration, which will be often necessary when large-scale Monte Carlo by quantum computer is in practical operation in the future.

II. OVERVIEW OF THE IDEA: QUANTUM ALGORITHM FOR MONTE CARLO USING PSEUDO-RANDOM NUMBER

A. Our Idea

Applications of the quantum algorithm for Monte Carlo to high-dimensional integration in financial problems can be found in previous works, especially credit risk measurement in [3]. In the paper, independent random numbers necessary to obtain a value of integrand are represented by different quantum registers, so the number of required qubits N_{qubit} is proportional to the number of random numbers N_{ran}. If N_{ran} is large as in the aforementioned cases, this can lead to shortage of qubits.

Let us see the method in more detail. The way to represent a random number by quantum register is as follows. For example, a qubit with state $\sqrt{1-p}\,|0\rangle + \sqrt{p}\,|1\rangle$ can be seen as a Bernoulli random number taking 1 with probability p. We can also represent a discretized approximation of a continuous random number like a normal random number on a quantum register [11]. Then, referring to these registers, the value of the integrand is computed into another register and its expectation value is estimated by methods such as [1, 8]. Note that this procedure intends to make a superposition of all possible patterns of random numbers[1] and the integrand value and estimate the *exact* expectation value, which we hereafter write as E_{true}.

In order to perform Monte Carlo enjoying quantum speedup and reducing qubits, we first note that what we calculate in classical Monte Carlo is different from that in the quantum way. That is, we do *not* consider *all patterns* of random number values in the classical Monte Carlo. We *sample only a part of patterns* of the random numbers and the integrand and take a simple arithmetic average of the sampled integrand values as an approximation for E_{true}. In other words, we calculate E_{samp}, the expectation value under the sample space which consists of a part of samples and the probability measure under which equal probability is allocated to each sample. Besides, in most cases, we use a sequence of PRN on behalf of random numbers to calculate the integrand, since strict randomness is difficult to realize on a classical computer. More concretely, we usually generate a PRN sequence with $N_{\mathrm{samp}}N_{\mathrm{ran}}$ elements and divide them into N_{samp} subsequences with N_{ran} elements, then use each subsequence to calculate a sample value of the integrand[2].

Our idea is that we estimate not E_{true} but E_{samp} using a quantum computer in the way similar to classical Monte Carlo. Before we describe the calculation flow in this method, let us state two assumptions necessary for it. The first assumption is that on the integrand. We assume that it takes N_{ran} random numbers as arguments and is sequentially computed in N_{ran} steps, each of which requires a random number and the output of the previous step as inputs. That is, using the intermediate functions $f_n, n = 1, ..., N_{\mathrm{ran}} - 1$, the value of the integrand $f_{N_{\mathrm{ran}}}$ for a given sequence $x_1, ..., x_{N_{\mathrm{ran}}}$ is calculated as

$$\begin{aligned} y_1 &= f_1(x_1), \\ y_n &= f_n(y_{n-1}, x_n) \quad \text{for} \quad n = 2, ..., N_{\mathrm{ran}}. \end{aligned} \tag{1}$$

We also assume that $f_{N_{\mathrm{ran}}}$ is normalized so that $0 \le f_{N_{\mathrm{ran}}}(y, x) \le 1$ for any x, y. Second, we assume that we can choose some PRNG which consumes n_{PRN} bits, including the random number itself and working space, and construct two types of quantum gate. One is P_{PRN}, which progresses a PRN sequence by one step, that is[3],

$$|x_n\rangle_{n_{\mathrm{PRN}}} \to |x_{n+1}\rangle_{n_{\mathrm{PRN}}}\,, \tag{2}$$

[1] If a continuous random number is approximated discretely, 'all patterns' means those of the discretized value.

[2] Mathematically, using such subsequences might raise a statistical concern for large N_{ran}, in terms of homogeneity of the distribution of tuples of consecutive PRNs in a high dimensional space [10]. However, such a way to use PRN is often adopted in practice in banks. We consider that using a tiny part in a large period PRN mitigates the concern [12].

[3] Here and hereafter, a subscript of a ket basically denotes the qubit number of the register.

where x_n is the n-th element of the PRN sequence. The other is J_{PRN}, which gives $x_{iN_{\mathrm{ran}}+1}$ for given i, that is,

$$|i\rangle_{n_{\mathrm{samp}}} |0\rangle_{n_{\mathrm{PRN}}} \rightarrow |i\rangle_{n_{\mathrm{samp}}} |x_{iN_{\mathrm{ran}}+1}\rangle_{n_{\mathrm{PRN}}} , \tag{3}$$

where n_{samp} is an integer which satisfies $0 < n_{\mathrm{samp}} < n_{\mathrm{PRN}}$. We show a concrete example of PRNGs which satisfies this assumption in section III.

Then, the calculation flow in our method is as follows. We take N_{samp}, the number of samples, as $N_{\mathrm{samp}} = 2^{n_{\mathrm{samp}}}$ for simplicity,

1. Prepare a register R_{samp} with n_{samp} qubits and generate a superposition of $|0\rangle, |1\rangle, ..., |N_{\mathrm{samp}} - 1\rangle$ with equal amplitudes, that is, $\frac{1}{\sqrt{N_{\mathrm{samp}}}} \sum_{i=0}^{N_{\mathrm{samp}}-1} |i\rangle_{n_{\mathrm{PRN}}}$. This can be done by operating a Hadamard gate to each of the n_{samp} qubits.

2. Operate J_{PRN}, then the $(iN_{\mathrm{ran}} + 1)$-th element of the sequence is set to the register R_{PRN}, where i is determined by the state of R_{samp}. These are the starting points of subsequences.

3. Perform a calculation step of the integral referring to R_{PRN} and reflect the result into a register R_{int}. Here, we assume that the integrand is calculated step-by-step using each random number.

4. Operate P_{PRN} to the register R_{PRN}, then the PRN sequence progresses by one step.

5. Perform a calculation step of the integral referring to R_{PRN} and reflect the result into a register R_{int}.

6. Iterate 4 and 5 until the calculation of the integrand ends. This corresponds to sequential generation of PRN and calculation using it. Finally, we obtain an equiprobable superposition of states, in each of which R_{int} holds a sampled integrand value.

7. Prepare an ancilla qubit, which we call R_{ph}, and encode the integrand value into the amplitude of $|1\rangle$ in R_{ph} using controlled rotations.

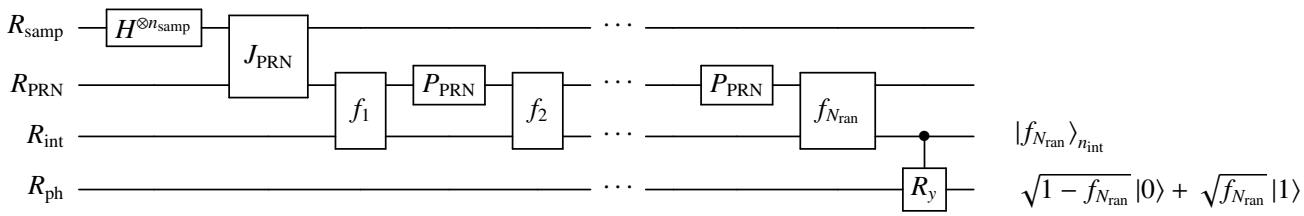
8. Estimate the amplitude of the state where R_{ph} is $|1\rangle$ by the amplitude estimation methods like [7, 8]. This is an estimate for the arithmetic average of sampled integrand values, that is, E_{samp}.

The flow of state transformation on $R_{\mathrm{samp}}, R_{\mathrm{PRN}}, R_{\mathrm{int}}$ and R_{ph} is as follows:

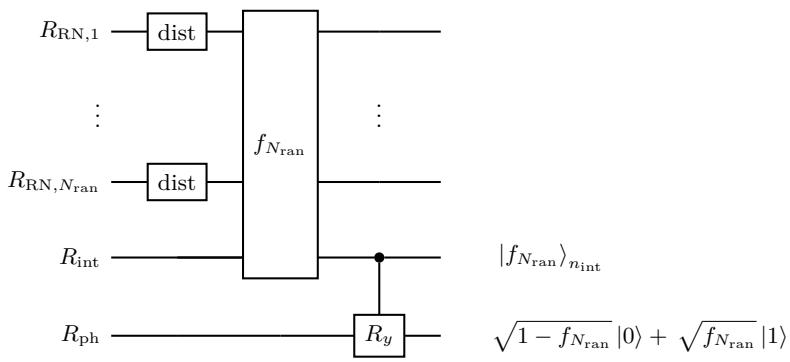
$$\begin{aligned}
&|0\rangle_{n_{\mathrm{samp}}} |0\rangle_{n_{\mathrm{PRN}}} |0\rangle_{n_{\mathrm{int}}} |0\rangle \\
&\xrightarrow{1} \frac{1}{\sqrt{N_{\mathrm{samp}}}} \sum_{i=0}^{N_{\mathrm{samp}}-1} |i\rangle_{n_{\mathrm{samp}}} |0\rangle_{n_{\mathrm{PRN}}} |0\rangle_{n_{\mathrm{int}}} |0\rangle \\
&\xrightarrow{2} \frac{1}{\sqrt{N_{\mathrm{samp}}}} \sum_{i=0}^{N_{\mathrm{samp}}-1} |i\rangle_{n_{\mathrm{samp}}} |x_1^{(i)}\rangle_{n_{\mathrm{PRN}}} |0\rangle_{n_{\mathrm{int}}} |0\rangle \\
&\xrightarrow{3} \frac{1}{\sqrt{N_{\mathrm{samp}}}} \sum_{i=0}^{N_{\mathrm{samp}}-1} |i\rangle_{n_{\mathrm{samp}}} |x_1^{(i)}\rangle_{n_{\mathrm{PRN}}} |f_1^{(i)}\rangle_{n_{\mathrm{int}}} |0\rangle \\
&\xrightarrow{4} \frac{1}{\sqrt{N_{\mathrm{samp}}}} \sum_{i=0}^{N_{\mathrm{samp}}-1} |i\rangle_{n_{\mathrm{samp}}} |x_2^{(i)}\rangle_{n_{\mathrm{PRN}}} |f_1^{(i)}\rangle_{n_{\mathrm{int}}} |0\rangle \\
&\xrightarrow{5} \frac{1}{\sqrt{N_{\mathrm{samp}}}} \sum_{i=0}^{N_{\mathrm{samp}}-1} |i\rangle_{n_{\mathrm{samp}}} |x_2^{(i)}\rangle_{n_{\mathrm{PRN}}} |f_2^{(i)}\rangle_{n_{\mathrm{int}}} |0\rangle \\
&\xrightarrow{6} ... \\
&\xrightarrow{6} \frac{1}{\sqrt{N_{\mathrm{samp}}}} \sum_{i=0}^{N_{\mathrm{samp}}-1} |i\rangle_{n_{\mathrm{samp}}} |x_{N_{\mathrm{ran}}}^{(i)}\rangle_{n_{\mathrm{PRN}}} |f_{N_{\mathrm{ran}}}^{(i)}\rangle_{n_{\mathrm{int}}} |0\rangle \\
&\xrightarrow{7} \frac{1}{\sqrt{N_{\mathrm{samp}}}} \sum_{i=0}^{N_{\mathrm{samp}}-1} |i\rangle_{n_{\mathrm{samp}}} |x_{N_{\mathrm{ran}}}^{(i)}\rangle_{n_{\mathrm{PRN}}} |f_{N_{\mathrm{ran}}}^{(i)}\rangle_{n_{\mathrm{int}}} \left(\sqrt{1 - f_{N_{\mathrm{ran}}}^{(i)}} |0\rangle + \sqrt{f_{N_{\mathrm{ran}}}^{(i)}} |1\rangle \right)
\end{aligned} \tag{4}$$

Here, $x_n^{(i)} = x_{iN_{\mathrm{ran}}+n}$ and this is the n-th element of the i-th subsequence. n_{int} is the qubit number of R_{int}. $f_1^{(i)}, .., f_{N_{\mathrm{ran}}}^{(i)}$ are the values of $f_1, ..., f_{N_{\mathrm{ran}}}$ for $x_1^{(i)}, ..., x_{N_{\mathrm{ran}}}^{(i)}$.

We present an outline of the quantum circuit for the above



(a)



(b)

Figure 1: Circuits for the quantum algorithm for Monte Carlo. Figures (a) and (b) correspond to the method we propose and that in previous papers, respectively. These are overviews and ancillas are not shown. Actually, the circuit for amplitude estimation follows the above circuits, but we omit it. See [7, 8] for the detail.

method in Figure 1. We also present that for the method in the previous papers for comparison. In our method, as shown in Figure 1a, after the operation which create a superposition of $|x_1^{(0)}\rangle, |x_1^{(1)}\rangle, ..., |x_1^{(N_{\mathrm{samp}}-1)}\rangle$ on R_{PRN} and the gate f_1, the first step of calculation of the integrand, we sequentially operate P_{PRN} and f_n, the n-th calculation step. The register which represents (pseudo) random numbers is only R_{PRN} and PRNs $x_n^{(i)}$ are sequentially generated on it. The intermediate value of the integrand $f_n^{(i)}$ is calculated into R_{int} using $x_n^{(i)}$ and $f_{n-1}^{(i)}$ as inputs and finally $f_{N_{\mathrm{ran}}}^{(i)}$ is reached. On the other hand, in the method in previous works, as shown in Figure 1b, quantum registers $R_{\mathrm{RN},1}, ..., R_{\mathrm{RN},N_{\mathrm{ran}}}$ are prepared to represent all random numbers simultaneously and a superposition of numbers following the desired probability distribution (for example, normal) is generated on each register by the gate 'dist' in Figure 1b. Then, the integrand value is calculated using all of $R_{\mathrm{RN},1}, ..., R_{\mathrm{RN},N_{\mathrm{ran}}}$ at the same time.

Here we make some comments. The first one is about the probability distribution of random numbers. In the previous method, a random number under the desired distribution is generated on a register using the gate 'dist'. On the other hand, in the method of this paper, sequentially generated PRNs basically obey uniform distribution, since most PRNGs are for that distribution. Therefore, we have to convert uniform random numbers to random numbers obeying a desired distribution. Such a conversion is actually a common step in the classical Monte Carlo and there are many well-known methods, for example, the Box-Muller method for standard normal distribution. We assume such a conversion is implementable as a quantum gate and contained in f_n. Actually, implementation of trigonometric functions and logarithm, which are necessary to the Box-Muller method, has been investigated in previous papers [13–15].

The second comment is about how the distribution of the integrand value is taken into account in the method of this paper. In the previous method, the desired distribution of the integrand value is realized through the distribution of random

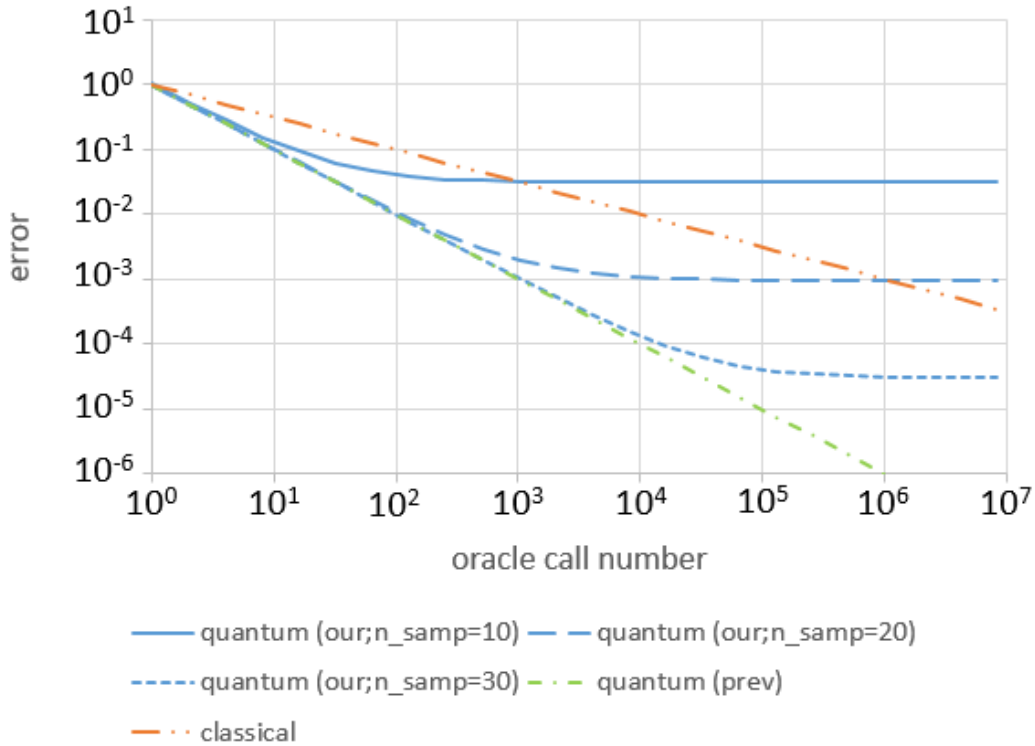


Figure 2: Errors in various methods for Monte Carlo. The blue solid, dashed and dotted lines are Δ_{our} in (5), the errors in the quantum method which we propose in this paper for various values of n_{samp}. The solid, dashed and dotted lines correspond to $n_{\text{samp}} = 10, 20$ and 30, respectively. The green chain line is Δ_{prev} in (11), the error in the quantum methods in the previous papers. The red two-dot chain line is Δ_{class} in (9), the error in the classical method. The horizontal axis is the oracle call number N_{orac}. We here set $c\sigma_{f_{N_{\text{ran}}}} = 1, 2\pi d\sqrt{E_{\text{samp}}(1-E_{\text{samp}})} = 1, 2\pi d\sqrt{E_{\text{true}}(1-E_{\text{true}})} = 1$.

numbers on registers. On the other hand, the distribution of the integrand value is generated through the PRNG in the method of this paper. Although the final state is a superposition of various integrand values with equal probability, the appearance pattern of the value reflects the distribution. For example, if the integrand value obeys distribution with a peak at some value F, the integrand values close to F frequently appear on R_{int} in the set of states that compose the final superposition.

Finally, we comment on the integrand form such that it can be computed sequentially. As mentioned above, it is just an assumption, that is, not all functions can be written like this. However, in many cases, the integrand takes this form; for examples of use cases of Monte Carlo, see textbooks such as [12]. We here give two frequent cases which satisfy (1) and include some important problems. First, the integrand takes the form of (1) if, after fixing some random numbers, we can write contributions from remaining random numbers in a separable sum or product, that is, $f_{N_{\text{ran}}}(x_1, ..., x_{N_{\text{ran}}}) = \sum_{j=d+1}^{N_{\text{ran}}} g_j(x_j; x_1, ..., x_d)$ or $f_{N_{\text{ran}}}(x_1, ..., x_{N_{\text{ran}}}) = \prod_{j=d+1}^{N_{\text{ran}}} g_j(x_j; x_1, ..., x_d)$, where d is a small natural number compared with N_{ran} and g_j, $j = d+1, ..., N_{\text{ran}}$ are some functions. The credit portfolio risk measurement, which we will consider later, corresponds to this case. Second, when we simulate Markov processes, they can be calculated in the sequential way like above. Pricing of financial derivative, where the underlying assets are Markov in many cases, is a typical example of this. Thinking of these examples, we are well motivated to consider the case where (1) is satisfied.

B. Relationship between Computational Load and Error

Now, we roughly estimate the relationship between computational load and additive error in three methods: the quantum method we propose, the quantum method proposed in previous papers and the classical method. In this paper, we measure computational load by N_{orac}, the number of times that we call the *oracle* in each method. Here, the *oracle* means the procedure to calculate the integrand. More concretely, it is the circuit (that in Figure 1, in the current case) and the subroutine to calculate the integrand for the quantum and classical method, respectively. In fact, in quantum methods, we have to repeatedly call the oracle circuit for amplitude estimation with the desired error level and this occupies the dominant part of the computation. On the other hand, the classical method requires the sufficient number of sampling to reduce the error and the computational time is almost proportional to the sample number.

First, let us consider our method. There are two sources of

error. One is the difference between E_{samp} and E_{true}, which we write as Δ_{TrSm}. The other is the estimation error of E_{samp}, that is, the error of amplitude estimation, which we write as Δ_{Est}. Δ_{TrSm} is equal to the statistical error in the classical Monte Carlo. For some fixed confidence level, it is at most $c\sigma_{f_{N_{\mathrm{ran}}}} N_{\mathrm{samp}}^{-1/2} = c\sigma_{f_{N_{\mathrm{ran}}}} 2^{-n_{\mathrm{samp}}/2}$, where $\sigma_{f_{N_{\mathrm{ran}}}}$ is the standard deviation of $f_{N_{\mathrm{ran}}}$ and c is a constant set according to the confidence level. Note that it depends on not N_{orac} but n_{samp}. On the other hand, Δ_{Est} is estimated as follows. The quantum algorithm in [7] with N_{orac} oracle calls gives estimation for the amplitude (that is, E_{samp}) which differs from the true value by at most $2\pi d\sqrt{E_{\mathrm{samp}}(1-E_{\mathrm{samp}})}N_{\mathrm{orac}}^{-1}$ with probability at least $1-\delta$. Here, we take only the leading term with respect to N_{orac}^{-1} and d is some $O(1)$ constant depending only on $\log\delta$. In total, the error in our method is at most

$$\begin{aligned}\Delta_{\mathrm{our}} &\sim \Delta_{\mathrm{TrSm}} + \Delta_{\mathrm{Est}} \\ &\simeq c\sigma_{f_{N_{\mathrm{ran}}}} 2^{-n_{\mathrm{samp}}/2} + 2\pi d\sqrt{E_{\mathrm{samp}}(1-E_{\mathrm{samp}})}N_{\mathrm{orac}}^{-1}. \quad (5)\end{aligned}$$

If we desire the error level ϵ, the following setting is sufficient. First, we set

$$N_{\mathrm{samp}} \sim \left(\frac{c\sigma_{f_{N_{\mathrm{ran}}}}}{\epsilon}\right)^2, \quad (6)$$

or, equivalently,

$$n_{\mathrm{samp}} \sim \lceil 2\log_2(c\sigma_{f_{N_{\mathrm{ran}}}}/\epsilon)\rceil, \quad (7)$$

so that $\Delta_{\mathrm{TrSm}} \sim \epsilon$. Then, we set

$$N_{\mathrm{orac}} \sim \frac{2\pi d\sqrt{E_{\mathrm{samp}}(1-E_{\mathrm{samp}})}}{\epsilon}, \quad (8)$$

which leads to $\Delta_{\mathrm{Est}} \sim \epsilon$. Note that N_{orac} does not depend on N_{samp}.

This is actually quadratic speed up compared with the classical Monte Carlo. In the classical method, the error is

$$\Delta_{\mathrm{class}} \sim c\sigma_{f_{N_{\mathrm{ran}}}} N_{\mathrm{orac}}^{-1/2}, \quad (9)$$

as Δ_{TrSm} in (5). Note that $N_{\mathrm{orac}} = N_{\mathrm{samp}}$ for the classical method. Then, the required N_{orac} in the method is

$$N_{\mathrm{orac}} \sim \left(\frac{c\sigma_{f_{N_{\mathrm{ran}}}}}{\epsilon}\right)^2 \quad (10)$$

for the desired error ϵ.

We also mention the error in the previous method of the quantum-based Monte Carlo. This method estimates E_{true} itself and the error is at most

$$\Delta_{\mathrm{prev}} \sim 2\pi d\sqrt{E_{\mathrm{true}}(1-E_{\mathrm{true}})}N_{\mathrm{orac}}^{-1}, \quad (11)$$

for the oracle call number N_{orac}. Note that this estimated error is nearly equal to Δ_{Est} in (5). This is because the error of amplitude estimation is determined by the amplitude itself and N_{orac} only [7] and the estimated amplitude is almost equal in both of the previous and our methods as long as Δ_{TrSm} is small.

Figure 2 represents the relationship among these errors in some specific case. We plot $\Delta_{\mathrm{our}}, \Delta_{\mathrm{prev}}$ and Δ_{class} versus N_{orac}. We here set the prefactors $c\sigma_{f_{N_{\mathrm{ran}}}}, 2\pi d\sqrt{E_{\mathrm{samp}}(1-E_{\mathrm{samp}})}, 2\pi d\sqrt{E_{\mathrm{true}}(1-E_{\mathrm{true}})}$ to 1. Besides, we set $n_{\mathrm{samp}} = 10, 20, 30$, which correspond to $N_{\mathrm{samp}} = 2^{10}(\approx 10^3), 2^{20}(\approx 10^6), 2^{30}(\approx 10^9)$, respectively. $N_{\mathrm{samp}} = 10^6$ is a typical value in the case of the credit risk measurement. When $\Delta_{\mathrm{TrSm}} \ll \Delta_{\mathrm{Est}}$, Δ_{our} decreases faster than Δ_{class} and similar to Δ_{prev} as N_{orac} increases. After Δ_{Est} becomes smaller than Δ_{TrSm}, Δ_{our} asymptotically converges to Δ_{TrSm}. However, for sufficiently large n_{samp}, Δ_{our} reaches the same order of magnitude as Δ_{class} for smaller N_{orac}. For example, when $n_{\mathrm{samp}} \geq 20$, Δ_{our} reaches the same order of magnitude as Δ_{class} for $N_{\mathrm{orac}} = 10^6$ only for $N_{\mathrm{orac}} = 10^3$, smaller by three orders of magnitude. In such a region, our method has an advantage compared to the classical way.

We also note that increasing n_{samp} by a few leads to increase of N_{samp} and decrease of Δ_{TrSm} by orders of magnitude. $N_{\mathrm{ran}}N_{\mathrm{samp}}$ cannot exceed P, the period of PRN, but we expect that it is unnecessary to concern such a upper bound, as long as we use a widely-used PRNG, which has a period, say 2^{64}. At least in the case of the credit risk measurement, this is much longer than $N_{\mathrm{ran}}N_{\mathrm{samp}}$ in practice, since each of these is at most 10^6 and the product is at most $10^{12} \sim 2^{40}$.

III. IMPLEMENTATION OF PSEUDO-RANDOM NUMBER GENERATOR ON QUANTUM CIRCUIT

A. PCG

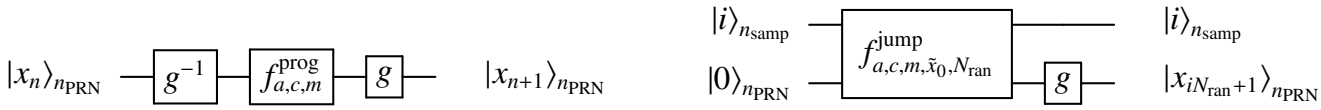
We next consider how to implement a PRNG on a quantum circuit, that is, the gates P_{PRN} and J_{PRN}. Remembering the motivation of this work, reduction of qubits, PRNGs which require small working space are desirable. Besides, in order to decrease circuit depth as much as possible, we desire a simpler and shorter calculation step to progress PRN sequence. Of course, the longer period and better statistical property is preferred. We propose PCG [9] as a PRNG which satisfies these properties.

PCG is combination of linear congruential generator (LCG), a popular and elementary PRNG, and permutation of bit string. The n-th element of a PCG sequence x_n is recursively defined as follows:

$$\begin{cases}\tilde{x}_{n+1} = f^{\mathrm{prog}}_{a,c,m}(\tilde{x}_n) := (a\tilde{x}_n + c) \mod m \\ x_n = g(\tilde{x}_n),\end{cases} \quad (12)$$

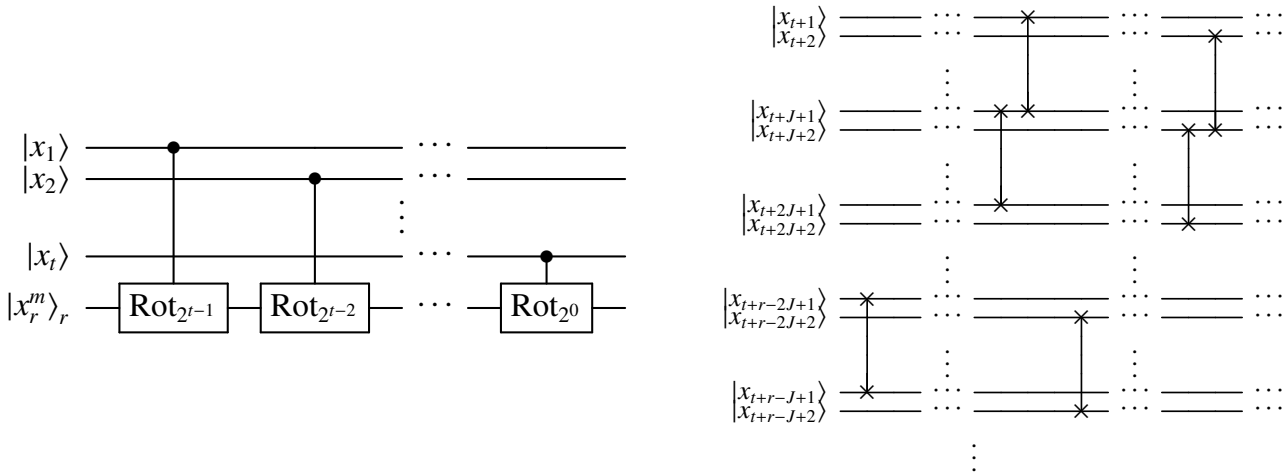
where a, c and m are integer parameters satisfying $a > 0, c \geq 0, m > 0$ and the seed $\tilde{x}_0$ is also given as an integer satisfying $0 \leq \tilde{x}_0 < m$. $\tilde{x}_n$ is the background sequence and defined by the LCG recurrence formula as above. g is the permutation of a bit string, which is explained in detail later. Therefore, the calculation steps to progress a PCG sequence is the sequence of modular multiplication, modular addition and permutation. Besides, for LCG we can easily jump ahead by k steps using the following formula:

$$\tilde{x}_{n+k} = \left(a^k\tilde{x}_n + \frac{c(a^k-1)}{a-1}\right) \mod m. \quad (13)$$



(a) (b)

Figure 3: Quantum gates for PCG. Figures (a) and (b) correspond to P_{PRN} and J_{PRN}, respectively.



(a) (b)

Figure 4: Quantum gate which performs random rotation. Figure (a) is the overview and (b) is the detail of Rot_J, $J = 2^j$.

Especially, we can obtain $\tilde{x}_{iN_{\mathrm{ran}}+1}$ from a seed x_0 as

$$\tilde{x}_{iN_{\mathrm{ran}}+1} = f^{\mathrm{jump}}_{a,c,m,\tilde{x}_0,N_{\mathrm{ran}}}(i) := \left(a^{iN_{\mathrm{ran}}+1}\tilde{x}_0 + \frac{c(a^{iN_{\mathrm{ran}}+1}-1)}{a-1}\right) \mod m. \tag{14}$$

Given the above formulas, we can construct quantum gates P_{PRN} and J_{PRN} for PCG. The rough images of the circuit diagrams are shown in Figure 3. To construct P_{PRN}, we first get back PCG to LCG with the inverse of g, then progress LCG with the $f^{\mathrm{prog}}_{a,c,m}$ gate and finally perform the permutation g. The $f^{\mathrm{prog}}_{a,c,m}$ gate maps $|x\rangle$ to $|f^{\mathrm{prog}}_{a,c,m}(x)\rangle$ and is constructed as modular multiplication $|x\rangle \to |ax \mod m\rangle$ followed by modular addition $|x\rangle \to |(x+c) \mod m\rangle$. To construct J_{PRN}, we first operate the $f^{\mathrm{jump}}_{a,c,m,\tilde{x}_0,N_{\mathrm{ran}}}$ gate, which refers to the first register as an input and transforms the second register from $|0\rangle$ to $|\tilde{x}_{iN_{\mathrm{ran}}+1}\rangle$ if the first register is $|i\rangle$, then g to the second register. The $f^{\mathrm{jump}}_{a,c,m,\tilde{x}_0,N_{\mathrm{ran}}}$ gate is constructed as a combination of modular addition, subtraction, multiplication, division and exponentiation $|k\rangle|x\rangle \to |k\rangle|a^k x \mod m\rangle$. Implementation of (modular) adder, multiplier and exponentiator has been investigated in many papers, for example, [16–27]. Modular subtraction is the inverse of addition. Division by $a-1$ modulo m is implemented as multiplication by an integer b such that $(a-1)b \equiv 1 \mod m$, which can be found by the extended Euclidean algo-

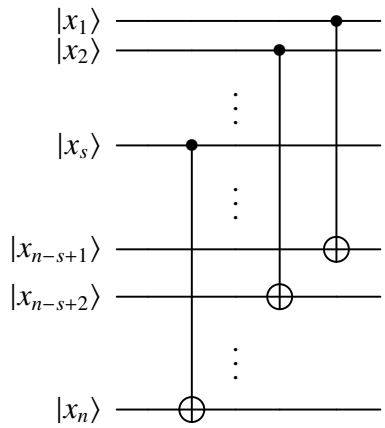


Figure 5: Quantum gate which performs xorshift.

rithm[4][28].

There is a comment on implementation of $f^{\text{prog}}_{a,c,m}$. It should be implemented not in the form that it output the result in the register other than the input register, that is, $|x\rangle|0\rangle \to |x\rangle|f^{\text{prog}}_{a,c,m}(x)\rangle$, but in the form that it updates the input register itself into the resulting state, that is, $|x\rangle \to |f^{\text{prog}}_{a,c,m}(x)\rangle$. This is because this gate is repeatedly used in the method of this paper, so the qubit number required for the entire calculation explodes if it is necessary to add a register in each calculation step. Most of the previous implementation of modular addition are the self-updating type, and so can be used with no change. On the other hand, some implementation of modular multiplication output the result into ancilla, $|x\rangle|0\rangle \to |x\rangle|ax \mod m\rangle$, but the trick described in [29] solves the problem as follows. First, using an integer a' such that $aa' \equiv 1 \mod m$, we construct a gate which performs $|x\rangle|0\rangle \to |x\rangle|a'x \mod m\rangle$ and its inverse. Then, we can implement the following sequence:

$$\begin{aligned} |x\rangle|0\rangle &\to |x\rangle|ax \mod m\rangle \\ &\to |ax \mod m\rangle|x\rangle \\ &\to |ax \mod m\rangle|0\rangle\,. \end{aligned} \tag{15}$$

Here, the first, second and third steps are modular multiplication by a, swap and the inverse of modular multiplication by a', respectively.

[4] Such b can be found if and only if $a-1$ and m are coprime. This condition is satisfied for many of widely used combination of a and m.

B. Permutation

It is well-known that LCG suffers from some statistical flaws. [9] points out that performing permutation on LCG enhances its statistical properties. Here, permutation is transformation of binary representation of a PRN to another bit string. We here take some of the permutations described in [9] as examples and show how to implement them in a quantum circuit.

The first one is random rotation. We first divide a n-bit binary $x \in \mathbb{Z}_{2^n}$ into three parts: the top t bits x^h_t, the middle r bits x^m_r and the bottom $n-t-r$ bits x^b_{n-t-r}, where r is a power of 2 and $t = \log_2 r$. Then random rotation is a map from $\mathbb{Z}_{2^n}$ to $\mathbb{Z}_{2^r}$ defined as

$$x \mapsto \sigma_{\text{rot}}(x^h_t, x^m_r). \tag{16}$$

Here,

$$\sigma_{\text{rot}}(k, y) := \begin{cases} y & ; k = 0 \\ y_{r-k+1}...y_r y_1...y_{r-k} & ; 1 \le k \le r-1 \end{cases} \tag{17}$$

for an integer k satisfying $0 \le k \le r-1$, $y = y_1 y_2 ... y_r \in \mathbb{Z}_{2^r}$ and $ab...$ represents a bit string whose first digit is $a \in \{0,1\}$, second digit is $b \in \{0,1\}$ and so on. In short, random rotation is clockwise rotation of middle digits of a binary where the rotation width is determined by the value of top digits. Only the middle digits x^m_r are used to calculate the integrand as a r bit random number. Especially, the bottom digits x^b_{n-t-r} are discarded since their statistical properties are not good.

Random rotation is easily implemented in a quantum circuit using controlled swap gate (Fredkin gate). The circuit diagram is shown in Figure 4. The middle bits $|x^m_r\rangle_r$ is rotated by 2^{t-i} bits by $\text{Rot}_{2^{t-i}}$ under the control of the top i-th bit, for $1 \le i \le t$. This leads to x^h_t-bit rotation of x^m_r. We can construct

the gate Rot_{2^j}, $j = 0, 1, ..., t-1$ connecting qubits with swap gates (actually Fredkin gates since these gates are controlled) as follows. Setting $J = 2^j$,

- Connect $|x_{t+(r/J-2)\cdot J+1}\rangle$ and $|x_{t+(r/J-1)\cdot J+1}\rangle$, $|x_{t+(r/J-3)\cdot J+1}\rangle$ and $|x_{t+(r/J-2)\cdot J+1}\rangle$,...,$|x_{t+1}\rangle$ and $|x_{t+J+1}\rangle$
- Connect $|x_{t+(r/J-2)\cdot J+2}\rangle$ and $|x_{t+(r/J-1)\cdot J+2}\rangle$,$|x_{t+(r/J-3)\cdot J+2}\rangle$ and $|x_{t+(r/J-2)\cdot J+2}\rangle$,...,$|x_{t+2}\rangle$ and $|x_{t+J+2}\rangle$
- ...
- Connect $|x_{t+(r/J-2)\cdot J+J}\rangle$ and $|x_{t+(r/J-1)\cdot J+J}\rangle$,$|x_{t+(r/J-3)\cdot J+J}\rangle$ and $|x_{t+(r/J-2)\cdot J+J}\rangle$,...,$|x_{t+J}\rangle$ and $|x_{t+J+J}\rangle$

That is, there are J groups containing n/J qubits connected by $n/J-1$ swap gates. Note that r/J is an integer.

The second type of permutation is xorshift. This is a map from $\mathbb{Z}_{2^n}$ to $\mathbb{Z}_{2^n}$ defined as follows:

$$\begin{aligned} x = x_1...x_n &\to x_1...x_{n-s}y_1...y_s, \\ y_i &\coloneqq x_i \oplus x_{n-s+i}, i = 1, ..., s. \end{aligned} \tag{18}$$

Here, s is an integer satisfying $1 \le s \le n-1$ and typically comparable with n, for example, $n/2$ as proposed in [9]. Note that we do not need to take xorshift over the whole qubits in the PRN register. That is, we can take XOR between top qubits and middle qubits and discard bottom ones, as random rotation.

We can construct a gate which performs this permutation using CNOT gates. That is, put NOT on $|x_{n-s+i}\rangle$ under control by $|x_i\rangle$, for $i = 1, ..., s$, as shown in Figure 5. Note the order to set CNOT gates, that is, from bottom to top. This is necessary in the case where $s > n/2$ so that some middle qubits are used as both a target and a control. Such a qubit must work as a control before it becomes a target.

C. Qubit Number and Circuit Depth

Here, we roughly estimate qubit number and depth of PCG circuits. We focus on P_{PRN}, which is repeatedly used to progress PRN sequences. We consider PCG with r-bit output and n-bit background LCG. n should be so large that the period, 2^n at most, is long enough and r is typically comparable with n. For example, in many of the settings considered in [9], $n = 64$ and $r = 32$.

The LCG part consists of modular addition and multiplication and dominant contribution comes from the latter. For n-bit operands, many of the proposed modular adder require $O(n)$ qubits including ancilla and $O(n)$ depth. On the other hand, modular multipliers basically require $O(n)$ qubits and $O(n^2)$ depth, so this is dominant in the LCG part[5].

The permutation part does not require any ancillas; at least two examples are mentioned above. Circuit depth is found as follows. For random rotation on r bits with $t = \log_2 r$ control bits, which we considered above, we first note that depth of swap gates in Rot_{2^j} is $r/2^j - 1$, since Rot_{2^j} consists of 2^j groups of $r/2^j$ qubits and $r/2^j - 1$ swap gates, as explained above. Summing this up for $j = 0, 1, ..., t-1$, it is found that the depth of Fredkin gates in the random rotation is $2r - \log_2 r - 2$, that is, $O(r)$. For xorshift with shift width s, it is obvious that the depth of CNOT gates is s and if s is comparable with r, say $s = r/2$ as considered in [9], so is the depth.

In summary, in terms of both ancilla qubit number and circuit depth, dominant contribution comes from a multiplication and is $O(n)$ and $O(n^2)$ respectively. Therefore, if each calculation step for integrand f_i contains computations heavier than several multiplications, PRN generation makes subdominant contributions to qubit number and circuit depth.

IV. APPLICATION TO CREDIT RISK MEASUREMENT

A. Merton Model

Now, let us consider the application of the aforementioned method to the actual problem in finance. We take credit risk measurement, which is mentioned in the introduction, as an example. First, we briefly explain the Merton model[6], which is widely used in practice in many banks.

In this model, the stochastic loss amount L in a credit portfolio consisting of N_{obl} obligors is given as follows:

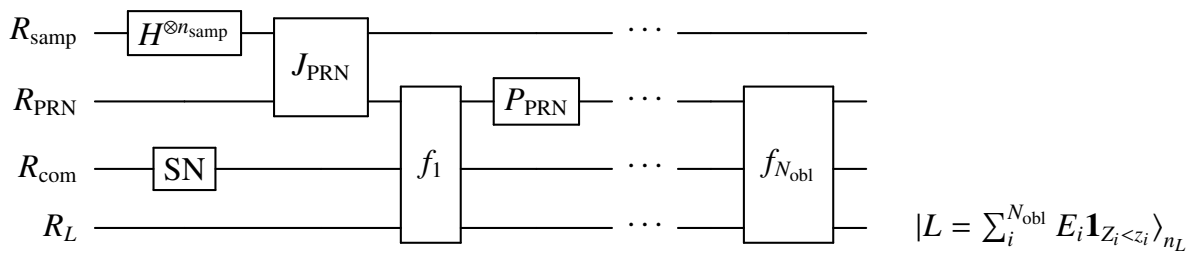
$$\begin{aligned} L &= \sum_i^{N_{\mathrm{obl}}} E_i \mathbf{1}_{Z_i < z_i}, \\ Z_i &= \alpha_i \epsilon_{\mathrm{com}} + \sqrt{1-\alpha_i^2}\epsilon_i. \end{aligned} \tag{19}$$

The meaning of each symbol is as follows. E_i is the exposure of the i th obligor, that is, the loss arising if he defaults[6]. $\mathbf{1}_C$ is the indicator function, which is 1 if the condition C is satisfied and 0 otherwise. The stochastic variable Z_i is interpreted as "the value of the firm" for the i th obligor. We consider that he defaults if Z_i becomes smaller than a threshold z_i. Usually, given a probability of default p_i exogenously, z_i is set as $z_i = \Phi_{\mathrm{SN}}^{-1}(p_i)$, where Φ_{SN} is the distribution function for standard normal distribution and Φ_{SN}^{-1} is its inverse. Z_i is given as a linear combination of two independent standard normal random variables ϵ_{com} and ϵ_i. ϵ_{com} is common for all obligors and called a systematic risk factor, which is interpreted as a factor reflecting the situation of macro economy[7]. ϵ_i is called an idiosyncratic risk factor and represents the effect of the matters unique to the i th obligor on his credit. We take the coefficient α_i such that $0 < \alpha_i < 1$; therefore, Z_i is also standard normal. α_i determines the correlation between Z_i for different obligors: the larger α_i means stronger correlation and a larger probability of simultaneous defaults of many obligors.

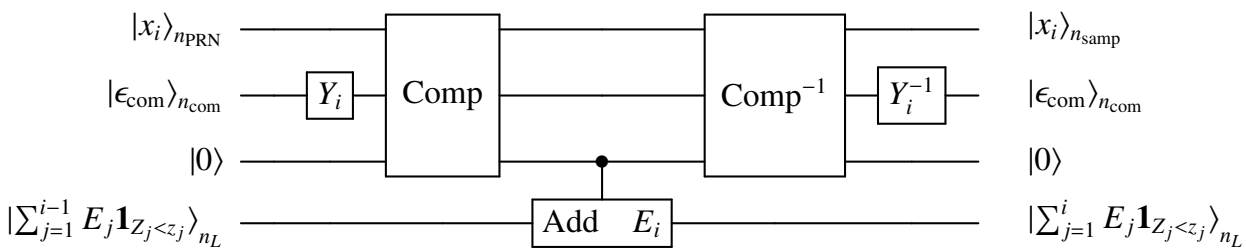
[5] There are implementations which have depth proportional to smaller powers of n than 2 but require ancilla proportional to larger powers of n than 1 [24].

[6] Here, we assume that loss given default is 1.

[7] Although we consider the case there is a single systematic risk factor, we can extend the model with multiple ones.



(a)



(b)

Figure 6: The Quantum circuit to calculate the loss amount in the Merton model. Figure (a) is the overview. Figure (b) is the detail of f_i. The first, second and fourth registers are $R_{\text{PRN}}, R_{\text{com}}$ and R_L respectively. The third register, to which the result of comparison is output, is omitted in (a).

B. Calculation of Loss Using PRNG on Quantum Circuit

Then, we describe how to calculate credit risk measures using a PRNG on a quantum circuit. What we have to develop is the circuit which calculates stochastic loss amount L. Once we develop the circuit which creates a superposition of states in which the value of the loss is encoded in some register, we can estimate VaR and CVaR as explained in [3]. The difference between the way in this paper and those in previous works is how to create such a superposition.

Seeing (19), we notice that the loss L can be written as a sum of contributions from each obligor and takes the form of (1) as mentioned above. More concretely, precomputing ϵ_{com} and defining

$$\begin{aligned} f_1(x) &= E_1 \mathbf{1}_{x<Y_1(\epsilon_{\text{com}})} \\ f_i(y, x) &= y + E_i \mathbf{1}_{x<Y_i(\epsilon_{\text{com}})}, i = 2, ..., N_{\text{obl}} \end{aligned} \tag{20}$$

we can calculate L as

$$\begin{aligned} y_1 &= f_1(x_1) \\ y_2 &= f_2(y_1, x_2) \\ &\vdots \\ y_{N_{\text{obl}}-1} &= f_{N_{\text{obl}}-1}(y_{N_{\text{obl}}-2}, x_{N_{\text{obl}}-1}) \\ L &= f_{N_{\text{obl}}}(y_{N_{\text{obl}}-1}, x_{N_{\text{obl}}}), \end{aligned} \tag{21}$$

using PRNs $x_1, ..., x_{N_{\text{obl}}}$ as $\epsilon_1, ..., \epsilon_{N_{\text{obl}}}$. Here, $Y_i(\epsilon_{\text{com}}) = M_{\text{PRN}} P_i(\epsilon_{\text{com}})$, where

$$P_i(\epsilon_{\text{com}}) = \Phi_{\text{SN}}\left(\frac{z_i - \alpha_i \epsilon_{\text{com}}}{\sqrt{1-\alpha_i^2}}\right). \tag{22}$$

is the conditional probability that the i-th obligor defaults given ϵ_{com} and M_{PRN} is the maximum number that the PRN can take.

The concrete calculation flow to obtain one sample value of L is as follows:

1. Generate a standard normal random variable and let it be $\epsilon_{\rm com}$.
2. Set $i = 1$ and $L = 0$.
3. Set the first elements of the PRN sequence x_1.
4. Calculate $Y_i(\epsilon_{\rm com})$.
5. Compare x_i with $Y_i(\epsilon_{\rm com})$. If the former is smaller than the latter, update $L \leftarrow L + E_i$.
6. If $i = N_{\rm obl}$, finish. Otherwise, progress the PRN sequence to get x_{i+1}, update $i \leftarrow i + 1$ and go to 4.

The above flow is performed by the circuit in Figure 6. As explained in Section II, we first create a superposition of $|x_1^{(1)}\rangle_{n_{\rm PRN}}, ..., |x_1^{(N_{\rm samp}-1)}\rangle_{n_{\rm PRN}}$ on $R_{\rm PRN}$ using $H^{\otimes n_{\rm samp}}$ and $J_{\rm PRN}$. These are starting elements of PRN sequences. Besides, we create a superposition of x, that is, numbers which obey the standard normal distribution in the register R_x. This is done by the method described in [11] and depicted as the gate 'SN' in Figure 6. Then, progressing the PRN sequence by $P_{\rm PRN}$, L is sequentially calculated by $f_1, ..., f_{N_{\rm obl}}$.

In f_i, at first, x is converted to $Y_i(x)$ by the gate Y_i[8]. We here simply assume that such a gate exists. In [2, 3], several ways to calculate such a function on a quantum computer are proposed, for example, linear approximation or piecewise polynomial approximation [13]. Then x_i is compared with $Y_i(x)$ and an ancillary qubit is set to 1 if $x_i > Y_i(x)$. Such a comparator has been presented in [30]. With the control by the ancilla, E_i is added to the loss register R_L. The controlled adder is also presented in previous papers, such as [16]. Finally, the inverses of Y_i and comparison are performed to uncompute R_x and the ancilla.

V. DEMONSTRATION: APPLICATION TO INTEGRATION OF SIMPLE MULTI-VARIABLE FUNCTION

Although the method proposed in this paper reduces required qubits, the circuit presented in the last section is still too large to perform in simulators or machines which can be publicly used today. We therefore consider a more small-scale problem performable in a simulator. It is an integral of a trigonometric function

$$I = \frac{1}{\theta^{N_{\rm var}}} \int_0^\theta dx_1 \cdots \int_0^\theta dx_{N_{\rm var}} \sin^2\left(\sum_{i=1}^{N_{\rm var}} x_i\right), \tag{23}$$

which is the multi-variable version of the problem considered in [8]. Note that the phase in the sin function depends on $N_{\rm var}$ variables. A naive way to calculate such a multi-variable integration numerically is discretization, that is, taking the sum of the integrand values on grid points set with equal interval in each axis,

$$I \simeq \frac{1}{N^{N_{\rm var}}} \sum_{i_1=0}^{N-1} \cdots \sum_{i_{N_{\rm var}}=0}^{N-1} \sin^2\left(\sum_{j=1}^{N_{\rm var}} \frac{i_j + 1/2}{N}\theta\right), \tag{24}$$

where N is the number of intervals in each axis.

However, in a brute force summation like this, the computational load increases exponentially with the number of variables since the number of grid points is $N^{N_{\rm var}}$. So the alternative way is Monte Carlo integration, that is, taking the average of the integrand values on grid points which are randomly sampled using PRNG. More specifically, taking an r-bit PRN sequence $\{x_i\}_{i=1,2,...}$, where $x_i \in \{0, 1, ..., 2^r - 1\}$, we make the approximation as

$$I \simeq \frac{1}{N_{\rm samp}} \sum_{i=0}^{N_{\rm samp}-1} \sin^2\left(\sum_{j=1}^{N_{\rm var}} \frac{x_j^{(i)} + 1/2}{2^r}\theta\right), \tag{25}$$

where $x_j^{(i)} = x_{iN_{\rm var}+j}$. Note that $(x_j^{(i)} + 1/2)/2^r$ is the pseudo-random number which takes one of 2^r grid points in an axis, $1/2^{r+1}, (1 + 1/2)/2^r, ..., (2^r - 1 + 1/2)/2^r$.

Note that we can use the method we proposed to calculate (25), since (25) is in the form of (1). Defining

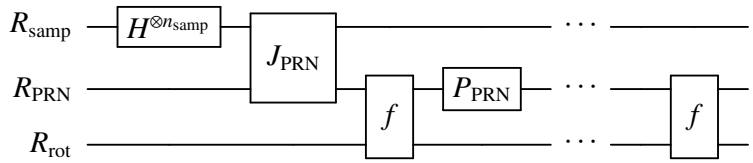
$$\begin{aligned} f_1(x) &= \frac{x + 1/2}{2^r}\theta \\ f_2(y, x) &= y + \frac{x + 1/2}{2^r}\theta \\ &\vdots \\ f_{N_{\rm var}-1}(y, x) &= y + \frac{x + 1/2}{2^r}\theta \\ f_{N_{\rm var}}(y, x) &= \sin^2\left(y + \frac{x + 1/2}{2^r}\theta\right), \end{aligned} \tag{26}$$

we can calculate a sample value of the integrand $f^{(i)} = \sin^2\left(\sum_{j=1}^{N_{\rm var}} \frac{x_j^{(i)}+1/2}{2^r}\theta\right)$ as

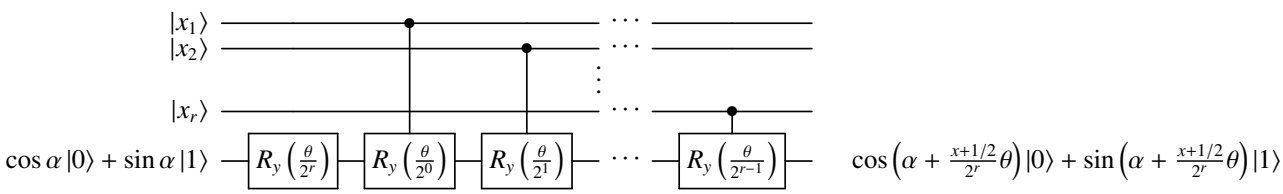
$$\begin{aligned} y_1^{(i)} &= f_1(x_1^{(i)}) \\ y_2^{(i)} &= f_2(y_1^{(i)}, x_2^{(i)}) \\ &\vdots \\ y_{N_{\rm var}-1}^{(i)} &= f_{N_{\rm var}-1}(y_{N_{\rm var}-2}^{(i)}, x_{N_{\rm var}-1}^{(i)}) \\ f^{(i)} &= f_{N_{\rm var}}(y_{N_{\rm var}-1}^{(i)}, x_{N_{\rm var}}^{(i)}). \end{aligned} \tag{27}$$

In fact, we perform the calculation in a slightly different way from (27), since there is a more efficient way in this case. The quantum circuit for the calculation is shown in Figure 7. In the circuit, we do not compute the integrand value on a register then rotate the phase of an ancilla with control of the register as (4), but sequentially rotate the ancilla's phase according to the PRN value. More concretely, the implementation is as follows. In addition to $R_{\rm samp}$ and $R_{\rm PRN}$, the circuit

[8] In practice, parameters such as p_i and α_i are not set for each obligor individually. Instead, obligors are grouped in terms of industry sector or rating and the same parameter values are given in obligors in each group. In such a case, it is not necessary to operate Y_i for each i, but sufficient to operate once per group.



(a)



(b)

Figure 7: The Quantum circuit for the integration (25). Figure (a) is the overview. Figure (b) is the detail of f. Here, $|x_1\rangle, ..., |x_r\rangle$ are qubits in R_{PRN} used as a r bit random number. $|x_1\rangle$ is the most significant digit and $|x_r\rangle$ is the least one.

(i) Exact value of the original integral	0.074578
(ii) Exact average of integrand values on sample points	0.078394
(iii) Estimate by the method in Section V	0.078391

Table I: Values of the integral obtained in various ways.

has an ancilla, which we hereafter write as R_{rot}. The value of the integration (25) is encoded into its phase by the gate f in Figure 7. This gate is a sequence of rotations around y-axis R_y controlled by output qubits in R_{PRN}[9]. That is, if R_{rot} is in the state $\cos\alpha\,|0\rangle + \sin\alpha\,|1\rangle$ for some real number α and R_{PRN} is in the state corresponding to a random number x before f, going through it transforms the state as follows:

$$\cos\alpha\,|0\rangle + \sin\alpha\,|1\rangle \to \cos\left(\alpha + \frac{x+1/2}{2^r}\theta\right)|0\rangle + \sin\left(\alpha + \frac{x+1/2}{2^r}\theta\right)|1\rangle\,, \quad (28)$$

that is, rotation by the angle $\frac{x+1/2}{2^r}\theta$. Therefore, starting from the state in which all registers are 0, the entire circuit transforms the state as follows:

$$|0\rangle_{\text{all}} := |0\rangle_{n_{\text{samp}}}\,|0\rangle_{n_{\text{PRN}}}\,|0\rangle \to \frac{1}{\sqrt{N_{\text{samp}}}}\sum_{i=1}^{N_{\text{samp}}} |i\rangle_{n_{\text{samp}}}\,|x^{(i)}_{N_{\text{var}}}\rangle_{n_{\text{PRN}}} \otimes\left[\cos\left(\sum_{j=1}^{N_{\text{var}}}\frac{x_j^{(i)}+1/2}{2^r}\theta\right)|0\rangle + \sin\left(\sum_{j=1}^{N_{\text{var}}}\frac{x_j^{(i)}+1/2}{2^r}\theta\right)|1\rangle\right]. \quad (29)$$

So the probability to observe $|1\rangle$ is equal to (25).

The probability to observe $|1\rangle$ can be estimated, for example, in the way proposed in [8], which we here explain briefly. First we construct the operation

$$Q = -AS_0A^{-1}S_\chi, \quad (30)$$

where A corresponds to the entire circuit in Figure 7, S_0 multiplies -1 to the state if all qubits are 0 or does nothing otherwise and S_χ multiplies -1 to the state if R_{rot} is 1 or does

[9] Note that not all qubits in R_{PRN} represent output random numbers. For example, bottom bits in PCG are not used due to poor statistical property.

nothing otherwise. If we write the probability to observe $|1\rangle$ in $R_{\rm rot}$ in $A|0\rangle_{\rm all}$ as $\sin^2\theta_a$, where $\theta_a \in [0, \pi/2]$, that in $|\Psi_m\rangle := Q^m A|0\rangle_{\rm all}$ is $\sin^2((2m+1)\theta_a)$. So, choosing a set of non-negative integers $m_0, m_1, ..., m_M$ and making N_k observations of $R_{\rm rot}$ in $|\Psi_{m_k}\rangle$ for each m_k, we can estimate θ_a as the maximum point of the following likelihood function:

$$L_{\rm lik}(\theta_a) := \prod_{k=0}^{M} \left[\sin^2((2m_k+1)\theta_a)\right]^{h_k} \left[\cos^2((2m_k+1)\theta_a)\right]^{N_k-h_k}, \tag{31}$$

where h_k is the number of observations where $R_{\rm rot}$ is $|1\rangle$ in $|\Psi_{m_k}\rangle$.

We have performed the actual calculation based on the above method using the quantum circuit simulator Qiskit of IBM [31]. The detailed setting is as follows. We estimate the integral (23) for $\theta = \pi/6$ and $N_{\rm var} = 2$. Although such a two-dimensional integral can be done analytically, the problem must be small-scale enough to be performed in the simulator and we consider it to be sufficient for a proof-of-concept. For PRNG, we use LCG with parameters $a = 11, c = 0, m = 31$ and the seed 1. Then the PRN is 5-bit and the period is 30. We take $N_{\rm samp} = 8$ sample points, so using 16 elements in the PRN sequence. Of course there are statistical concerns on the estimate based on such a small number of samples generated by such a small-scale PRNG, but it is inevitable in calculation on a simulator and sufficient for the current proof-of-concept purpose. If we can use a real quantum computer with sufficient qubits, say 100, we should use PCG under an appropriate setting: with sufficiently many qubits (say, 32-bit output and 64-bit background LCG), widely-used LCG parameters and permutation recommended in [9]. For the implementation of LCG, we use the adder presented in [27] and construct modular adder, multiplier and exponentiator based on the adder following the way in [16]. For θ_a estimation, we take $M = 8, N_k = 100$ and $m_k = 2^k$, as in [8].

We show the result in Table I. At the time when the integral (23) is approximated as (25), some error arises. This is the difference between (i) and (ii) in Table I, which will become smaller if we can take more sample points generated by a larger-scale PRNG. Estimation by quantum computer should converge to (ii), and the estimation obtained actually (iii) is close to (ii) as expected.

VI. CONCLUSION AND DISCUSSION

In this paper, we considered reduction of a qubit number in the quantum algorithm for Monte Carlo. Although its applications to problems in finance are proposed in previous works, high-dimensionality of some of such problems requires many qubits if a quantum register is prepared for each of the random numbers required to calculate one sample value of the integrand. Especially, for credit risk measurement, the required qubit number is proportional to the number of obligors, which can be $O(10^6)$. Then we proposed a way to reduce the qubit number. Considering the difference between what we calculate in the previous way of quantum-based Monte Carlo and that in classical Monte Carlo, we pointed out that estimating the average of sampled integrand values, which is calculated in classical Monte Carlo, by the quantum algorithm provides us with both quantum speed-up and qubit reduction. We saw that such a way is realized by the PRNG on a quantum computer and presented a candidate for a PRNG implementable on a quantum computer, PCG, with concrete circuit diagrams. We also described how to implement credit risk measurement using PRNG on quantum computer and demonstrated a simple integral on a quantum circuit simulator as a proof-of-concept.

As a final note, let us consider the trade-off between qubit number and circuit depth. It is clear that qubit number reduction proposed in this paper increases circuit depth. It is change of the design of the circuit, from that parallelly generate random numbers in different registers to that sequentially generate them in a register[10]. Therefore, circuit depth is now proportional to the number of random numbers $N_{\rm ran}$[11]. This might make full reduction of qubit number by this way impractical. Without quantum error correction [32–34], which is expected not to be realized in near-term quantum computer, such a deep circuit will not be performable. Even if a machine with error correction is developed, deep circuits might suffer from long runtime of fault-tolerant gates [3, 35] and quantum computation with small computational *load* might not necessarily lead to short computational *time*.

However, we consider the above trade-off itself meaningful. Even if a quantum computer with large qubit number becomes in operation in the future, management of memory (that is, qubit) will be an important issue when it is applied to large-scale problems such as credit risk measurement. That is, when fully parallel computation is impossible due to shortage of memory, we have to perform some procedures in sequence. This is an issue which frequently arises also in today's classical computation.

The method proposed in this paper provides a way to solve such a problem in large-scale Monte Carlo simulation by quantum computer. Consider the situation where $N_{\rm ran}$ random numbers are required to calculate the integrand but the available machine has so small a number of qubits that only $N_{\rm ran}/n$ random numbers can be generated at the same time, where n is an integer satisfying $n \geq 2$. In such a case, we can parallelly generate $N_{\rm ran}/n$ PRN sequences with n elements, calculate a part of the integrand using the elements in each sequence one-by-one, and finally merge partial results to obtain the entire integrand value[12]. This leads to the circuit depth proportional to n. This is partial but maximum parallelism which can be done in the machine, although the depth is n times larger than the full parallelism.

---

[10] Here, "parallel" means not parallel computation in quantum superposition but that in separate memories, which is possible also in classical computers.

[11] Note that, depending on problems, circuit depth can be proportional to $N_{\rm ran}$ even if random numbers are generated on different registers. That is, if there is no other way than calculating the integrand using random numbers in sequence, circuit depth is inevitably $O(N_{\rm ran})$, whether we generate random numbers sequentially or parallelly. Calculation of loss in a credit portfolio can be parallelized, as explained in [3].

[12] Again, this is possible only if the integrand allows such calculation.

ACKNOWLEDGEMENTS

The authors thank Shumpei Uno of Mizuho Information & Research Institute and Kazuyoshi Yoshino, Naoyuki Takeda and Kazuya Kaneko of Mizuho-DL Financial Technology for helpful comments.

[1] A. Montanaro, Quantum speedup of Monte Carlo methods, Proc. Roy. Soc. Ser. A, 471, 2181 (2015)
[2] S. Woerner et al., Quantum risk analysis, npj Quantum Information 5, 15 (2019)
[3] D. J. Egger et al., "Credit Risk Analysis using Quantum Computers", arXiv:1907.03044
[4] P. Rebentrost et al., "Quantum computational finance: Monte Carlo pricing of financial derivatives", Phys. Rev. A 98, 022321 (2018)
[5] N. Stamatopoulos et al., "Option Pricing using Quantum Computers", Quantum 4, 291 (2020).
[6] R. C. Merton, "On the pricing of corporate debt: The risk structure of interest rates", J. Finance, 29, 449 (1974)
[7] G. Bassard et al., Quantum amplitude amplification and estimation, Contemporary Mathematics, 305, 53 (2002)
[8] Y. Suzuki et al., Amplitude Estimation without Phase Estimation, arXiv:1904.10246
[9] M. E. ONeill, PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation, Harvey Mudd College Computer Science Department Tachnical Report (2014); http://www.pcg-random.org/
[10] M. Matsumoto et al., "Mersenne Twister: A 623-dimensionally Equidistributed Uniform Pseudo-random Number Generator", ACM Transactions on Modeling and Computer Simulation (TOMACS) 8, 1 , 3 (1998)
[11] L. Grover et al., Creating superpositions that correspond to efficiently integrable probability distributions, arXiv:quant-ph/0208112
[12] P. Glasserman, "Monte Carlo Methods in Financial Engineering", Springer (2003)
[13] T. Haner et al., "Optimizing Quantum Circuits for Arithmetic", arXiv:1805.12445
[14] Y. Cao et al., "Quantum algorithm and circuit design solving the Poisson equation", New J. Phys. 15, 013021 (2013)
[15] M. K. Bhaskar et al., "Quantum Algorithms and Circuits for Scientific Computing", Quantum Information and Computation, 16(3&4), 0197 (2016)
[16] V. Vedral et al., Quantum Networks for Elementary Arithmetic Operations, Phys. Rev. A 54, 147 (1996)
[17] D. Beckman et al., "Efficient networks for quantum factoring", Phys. Rev. A, 54, 1034 (1996)
[18] T. G. Draper, "Addition on a quantum computer", arXiv:quant-ph/0008033
[19] S. A. Cuccaro et al., "A new quantum ripple-carry addition circuit", arXiv:quant-ph/0410184
[20] Y. Takahashi et al., "A linear-size quantum circuit for addition with no ancillary qubits", Quantum Information and Computation, 5(6), 440448 (2005)
[21] R. Van Meter et al., "Fast quantum modular exponentiation", Phys. Rev. A 71(5), 052320 (2005)
[22] T. G. Draper et al., "A logarithmic-depth quantum carry-lookahead adder", Quantum Information and Computation, 6(4), 351 (2006)
[23] Y. Takahashi et al., "Quantum addition circuits and unbounded fan-out", Quantum Information and Computation, 10(9&10), 0872 (2010)
[24] L. A. B. Kowada, R. Portugal and C. M. H. Figueiredo, "Reversible Karatsubas algorithm", J. Univ. Comput. Sci. 12, 499 (2006).
[25] J. J. Alvarez-Sanchez et al., "A quantum architecture for multiplying signed integers", Journal of Physics: Conference Series, 128(1), 012013 (2008)
[26] Y. Takahashi et al., "A fast quantum circuit for addition with few qubits", Quantum Information and Computation, 8(6), 636 (2008)
[27] H. Thapliyal et al., Design of Efficient Reversible Logic Based Binary and BCD Adder Circuits, J. Emerg. Technol. Comput. Syst. 9, 17, 1 (2013)
[28] D. E. Knuth, "The Art of Computer Programming, Volume 2: Seminumerical Algorithms", Addison-Wesley (1981)
[29] I. L. Markov et al., Constant-Optimized Quantum Circuits for Modular Multiplication and Exponentiation, Quantum Information and Computation, 12(5&6), 0361 (2012)
[30] D. S. Oliveira and R. Ramos, Quantum bit string comparator: circuits and applications, Quantum Computers and Computing 7, 17 (2007).
[31] G. Aleksandrowicz et al. Qiskit: An open-source framework for quantum computing (2019); https://qiskit.org/
[32] P. W. Shor, Fault-tolerant quantum computation, in Proceedings of the 37th Annual Symposium on Foundations of Computer Science, FOCS 96 (IEEE Computer Society, Washington, DC, USA, 1996) pp. 56
[33] A. Y. Kitaev, Fault-tolerant quantum computation by anyons, Annals of Physics 303, 2 (2003)
[34] A. G. Fowler et al., Surface codes: Towards practical large-scale quantum computation", Phys. Rev. A 86, 032324 (2012)
[35] A. G. Fowler et al., Low overhead quantum computation using lattice surgery, arXiv:1808.06709